GUIA TÉCNICO | 12

Persistência de modelos e reprodutibilidade

Capítulo 12/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Diferenciar serialização de modelo, pesos e pipeline completo.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

O formato Keras moderno .keras salva configuração, pesos e estado do otimizador quando aplicável. Arquivos .h5 históricos requerem compatibilidade. Preservar versão das dependências, transformações, sementes, métricas e esquema de dados. Nunca carregar modelos de fontes não confiáveis.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: artefato = arquitetura + pesos + estado

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Persistência inclui modelo, pré-processamento, versões e comportamento após recarga; carregamentos inseguros podem executar código.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import json, numpy as np
metadata={'seed':42,'split':'estratificado','metrica':'f1','framework':'registro  ↪
    didático'}
encoded=json.dumps(metadata,ensure_ascii=False)
assert json.loads(encoded)['seed']==42
print('Manifesto de experimento:',encoded)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um

conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Persistência, restauração e identidade das previsões

Persistência inclui modelo, pré-processamento, versões e comportamento após recarga; carregamentos inseguros podem executar código.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import tempfile
from pathlib import Path
import joblib,numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
X=np.array([[0,0],[0,1],[1,0],[1,1],[2,0],[0,2]],dtype=float)
y=np.array([0,0,0,1,1,1]);m=make_pipeline(StandardScaler(),LogisticRegression()).fit(X,y)
with tempfile.TemporaryDirectory() as d:
    fp=Path(d)/'modelo.joblib';joblib.dump(m,fp);rec=joblib.load(fp)
    assert np.allclose(m.predict_proba(X),rec.predict_proba(X))
    print('Modelo restaurado, probabilidades idênticas, ↪
    tamanho:',fp.stat().st_size,'bytes')
```

Nunca carregue modelos serializados de fonte não confiável. Registre versão das bibliotecas, parâmetros de treino e métricas além do artefato.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Salvar apenas pesos basta para reproduzir o pipeline?

Resposta: Não; também são necessários arquitetura, pré-processamento, hiperparâmetros e versões.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide